

Assert.js v1.1.5 cheatsheet - <https://github.com/Serrin/assert.js>

Import assert	<pre>import assert from "./assert.js"; globalThis.assert = assert;</pre>
Import assert as default	<pre>import { default as assert } from "./assert.js"; globalThis.assert = assert;</pre>
Import assert as defaultExport	<pre>import defaultExport from "./assert.js"; globalThis.assert = defaultExport;</pre>
Dynamic import	<pre>const assert = await import("./assert.js"); globalThis.assert = assert;</pre>
Config	<pre>assert.config.alwaysStrict;</pre> <i>Default value is false. If true, then `assert.equal();` will be replaced with `assert.strictEqual();` and `assert.notEqual();` will be replaced with the `assert.notStrictEqual();`.</i>
Testrunner functions	<pre>type TResult<T> = {ok: true, value: T, block: Function, name: string} {ok: false, error: Error, block: Function, name: string}; assert.testSync(block, name = "assert.testSync"): TResult await assert.testAsync(block, name = "assert.testAsync"): TResult assert.testCheck(result: TResult): result.ok is true</pre>

Category	Assertions	
Constants	assert.VERSION;	
Errors	assert.AssertionError([message], [options]);	
Basic	assert(condition, [message: string Error]); assert.ok(condition, [message: string Error]); assert.notOk(condition, [message: string Error]); assert.fail([message: string Error]);	
Equality	assert.equal(actual, expected, [message: string Error]);	<i>loose equality: `==`</i>
	assert.notEqual(actual, expected, [message: string Error]);	<i>loose inequality: `!=`</i>
	assert.strictEqual(actual, expected, [message: string Error]);	<i>uses `Object.is();`</i>
	assert.notStrictEqual(actual, expected, [message: string Error]);	<i>uses `!Object.is();`</i>
	assert.deepEqual(actual, expected, [message: string Error]);	<i>uses `Object.is();`</i>
	assert.notDeepEqual(actual, expected, [message: string Error]);	<i>uses `!Object.is();`</i>
	assert.oneOf(value, collection, [message: string Error]);	<i>uses `Object.is();`</i>
	assert.notOneOf(value, collection, [message: string Error]);	<i>uses `!Object.is();`</i>
Exceptions	assert.throws(fn, [ErrorType string RegExp], [message: string Error]): Promise; await assert.rejects(asyncFnOrPromise, [ErrorType string RegExp], [message: string Error]); await assert.doesNotReject(asyncFnOrPromise, [ErrorType string RegExp], [message: string Error]);	
Boolean	assert.isTrue(value, [message: string Error]); assert.isNotTrue(value, [message: string Error]); assert.isFalse(value, [message: string Error]); assert.isNotFalse(value, [message: string Error]);	
String	assert.match(string, regexp, [message: string Error]); assert.doesNotMatch(string, regexp, [message: string Error]); assert.stringContains(actual, substring, [message: string Error]); assert.stringNotContains(actual, substring, [message: string Error]);	
Objects	assert.includes(container, options: {keyOrValue, [value] }, [message: string Error]); assert.doesNotInclude(container, options: {keyOrValue, [value] }, [message: string Error]); assert.isEmpty(value, [message: string Error]); assert.isNotEmpty(value, [message: string Error]);	

Comparison	<code>assert.lt(value1, value2, [message: string Error]);</code> <code>assert.lte(value1, value2, [message: string Error]);</code> <code>assert.gt(value1, value2, [message: string Error]);</code> <code>assert.gte(value1, value2, [message: string Error]);</code> <code>assert.inRange(value, min, max, [message: string Error]);</code> <code>assert.notInRange(value, min, max, [message: string Error]);</code>	
Type	<code>assert.is(value, expectedType: string function Array<string function>, [message: string Error]);</code> <code>assert.isNot(value, expectedType: string function Array<string function>, [message: string Error]);</code>	
	<code>assert.isPrimitive(value, [msg: string Error]);</code>	<code>assert.isNotPrimitive(value, [msg: string Error]);</code>
	<code>assert.isNullish(value, [msg: string Error]);</code>	<code>assert.isNotNullable(value, [msg: string Error]);</code>
	<code>assert.isNull(value, [message: string Error]);</code>	<code>assert.isNotNull(value, [message: string Error]);</code>
	<code>assert.isUndefined(value, [msg: string Error]);</code>	<code>assert.isDefined(value, [message: string Error]);</code>
	<code>assert.isNumber(value, [msg: string Error]);</code>	<code>assert.isNotNumber(value, [msg: string Error]);</code>
	<code>assert.isNaN(value, [message: string Error]);</code>	<code>assert.isNotNaN(value, [message: string Error]);</code>
	<code>assert.isInt(value, [message: string Error]);</code>	<code>assert.isNotInt(value, [message: string Error]);</code>
	<code>assert.isFloat(value, [message: string Error]);</code>	<code>assert.isNotFloat(value, [message: string Error]);</code>
	<code>assert.isString(value, [msg: string Error]);</code>	<code>assert.isNotString(value, [msg: string Error]);</code>
	<code>assert.isBigInt(value, [msg: string Error]);</code>	<code>assert.isNotBigInt(value, [msg: string Error]);</code>
	<code>assert.isBoolean(value, [msg: string Error]);</code>	<code>assert.isNotBoolean(value, [msg: string Error]);</code>
	<code>assert.isSymbol(value, [msg: string Error]);</code>	<code>assert.isNotSymbol(value, [msg: string Error]);</code>
	<code>assert.isFunction(value, [msg: string Error]);</code>	<code>assert.isNotFunction(value, [msg: string Error]);</code>
	<code>assert.isObject(value, [msg: string Error]);</code>	<code>assert.isNotObject(value, [msg: string Error]);</code>